

## Módulo 10 – Capítulo 04 – Sección 01

---

### Feature store: repositorio centralizado de features para modelos de ML y IA

Este capítulo cubre dos dominios de gestión de datos que son distintos pero frecuentemente confundidos: los feature stores para modelos de ML clásico (tabular, recomendación, clasificación) en las primeras cuatro secciones, y los pipelines de datos especializados para LLMs (corpus de preentrenamiento, datos de instruction tuning) en las secciones 03 y 04. Las herramientas, problemas y arquitecturas de cada dominio son parcialmente independientes: Feast o Tecton no aparecen en un pipeline de preprocesamiento de Common Crawl para preentrenamiento de LLMs, y MinHash LSH no tiene relevancia en un feature store de comportamiento de usuarios. Identificar en qué dominio se trabaja antes de elegir herramientas evita la confusión habitual de aplicar soluciones de un dominio al problema del otro.

Un **feature store** resuelve dos problemas críticos de los proyectos de ML a escala. El primero es la duplicación de lógica de transformación de features entre equipos: sin un repositorio central, el equipo de recomendaciones calcula "tiempo\_desde\_último\_purchase" con una implementación en pandas, el equipo de fraude la recalcula con una implementación ligeramente diferente en PySpark, y el equipo de pricing usa una tercera versión en SQL. Las tres implementaciones producen resultados similares pero no idénticos, lo que significa que ningún equipo puede reutilizar el trabajo de los demás con confianza. El feature store permite que la lógica de cálculo se defina una vez con un contrato explícito (nombre, tipo, descripción, owner, TTL) y se reutilice por todos los modelos que la necesiten, eliminando la posibilidad de divergencia.

El segundo problema que resuelve el feature store es el **training-serving skew**: la diferencia entre las features disponibles en entrenamiento y las disponibles en inferencia en producción. Este es uno de los bugs más silenciosos y peligrosos en sistemas de ML: el modelo se entrena con una feature calculada sobre el historial completo de datos, pero en producción esa misma feature se calcula con una lógica ligeramente diferente o con datos de un sistema diferente, produciendo una distribución de valores distinta que el modelo no vio en

entrenamiento. El training-serving skew puede degradar la calidad de los modelos de forma significativa sin producir ningún error de sistema: el endpoint responde correctamente, pero las predicciones son silenciosamente peores. El feature store elimina este problema definiendo una única implementación de cada feature que se ejecuta tanto en el contexto de entrenamiento (offline) como en el contexto de inferencia (online).

La arquitectura de un feature store se divide en dos almacenes complementarios. El **almacén offline** —Parquet sobre S3, Delta Lake en Databricks, o BigQuery en GCP— contiene el historial completo de features con point-in-time correctness: la capacidad de recuperar el valor exacto que cada feature tenía en un momento específico del pasado, sin contaminar el set de entrenamiento con información futura que el modelo no habría tenido disponible en producción (data leakage). El **almacén online** —Redis Cluster, DynamoDB, o Bigtable— contiene la versión más reciente de cada feature con latencias de sub-milisegundo para inferencia en tiempo real: cuando una petición de inferencia llega, el serving layer recupera las features del usuario del almacén online en menos de 10ms y las pasa al modelo. La **materialización** es el proceso de pipeline que calcula las features desde las fuentes de datos originales, las escribe al almacén offline, y las sincroniza periódicamente al almacén online según la frecuencia definida (cada hora, diariamente, en streaming continuo).

## Componentes principales de un feature store

- **Feature definitions:** especificación declarativa de cada feature con nombre, tipo (float32, int64, string), descripción, owner, TTL en el almacén online, y transformación de cálculo; el contrato explícito que garantiza reproducibilidad.
- **Offline store:** almacén columnar con historial temporal completo; soporta point-in-time joins para construir training sets sin data leakage usando el timestamp exacto de cada evento de entrenamiento.
- **Online store:** almacén de baja latencia (Redis, DynamoDB) para inferencia en tiempo real; sincronizado automáticamente desde el offline store por un job de materialización programado en el orquestador de pipelines.
- **Feature registry:** catálogo de todas las features disponibles con metadatos, ejemplos y estadísticas de distribución; el punto de descubrimiento que permite a los equipos encontrar features existentes antes de implementar nuevas que ya existen.
- **Materialización:** proceso de pipeline que calcula y escribe las features desde las fuentes de datos al offline store, y luego las sincroniza al online store; la frecuencia de materialización determina la frescura de las features online.

**Nota del Arquitecto:** *El valor de un feature store es fundamentalmente organizacional antes de ser técnico. La reducción del training-serving skew es valiosa, pero el beneficio más duradero es que detiene la proliferación de implementaciones paralelas de las mismas features en equipos distintos. El ROI del feature store se mide en tiempo ahorrado de descubrimiento (¿ya existe una feature de comportamiento del usuario?), en tiempo ahorrado de debugging (¿por qué mi modelo en producción es peor que en training?) y en confianza aumentada en los resultados de los experimentos.*

El feature store es el componente de la plataforma que garantiza que los modelos se entrenan con los datos correctos y que la inferencia usa las mismas features que el entrenamiento. La siguiente sección compara las tres implementaciones más adoptadas —Feast, Hopsworks y Tecton— con criterios de selección que van más allá de las comparaciones de features en papel para llegar a los factores que determinan el éxito en la práctica.

---

## Módulo 10 – Capítulo 04 – Sección 02

---

### Feast, Hopsworks y Tecton: comparación de soluciones de feature store

Elegir un feature store no es solo una decisión técnica: es una decisión sobre cuánto esfuerzo operativo el equipo de plataforma está dispuesto a asumir a cambio de cuánta flexibilidad y cuánto control. Las tres plataformas más adoptadas en la industria —Feast, Hopsworks y Tecton— tienen posicionamientos suficientemente distintos como para que la elección correcta varíe significativamente según el contexto de la organización. Comparar solo sus listas de features en papel produce conclusiones incorrectas; la comparación útil es la que considera el esfuerzo operativo real, la curva de aprendizaje del equipo, el costo total de ownership, y el tipo de casos de uso que dominan en la organización.

**Feast** (Feature Store, de Gojek, ahora mantenido por la Linux Foundation con contribuciones de Google, Twitter y Tecton) es el feature store open source de referencia y el punto de partida más común para organizaciones que quieren control total sin costo de licencia. Su modelo de programación es declarativo: las features se definen en Python con `FeatureView`, `Entity` y `FeatureService`, y Feast gestiona la materialización al almacén online y las consultas point-in-time para el training. La flexibilidad de Feast viene con un costo: el equipo de plataforma debe gestionar la infraestructura subyacente (Redis para el online store, S3 para el offline store, y el orquestador de materialización). Esta responsabilidad es manejable para equipos con experiencia en Kubernetes que ya operan múltiples servicios, pero puede ser un obstáculo significativo para equipos más pequeños o menos especializados en infraestructura. Feast es la opción correcta cuando la organización tiene ya capacidad operativa de Kubernetes, quiere control total sobre los backends de almacenamiento, y tiene casos de uso batch con ciclos de materialización de horas a días.

**Hopsworks** añade sobre el modelo de Feast un motor de ingesta streaming integrado (Flink) que permite features calculadas en tiempo real con latencias de segundos, feature versioning nativo con `FeatureGroup.insert()` que mantiene el historial de cada versión de las definiciones de features, y una UI rica con estadísticas automáticas de distribución calculadas en cada escritura al feature group. La versión managed (Hopsworks AI en AWS, GCP o Azure)

elimina la carga operativa del despliegue y mantenimiento de la infraestructura subyacente, a un costo de licencia que escala con el volumen de datos procesados. Hopsworks es la opción correcta para organizaciones que tienen casos de uso de streaming features (features que requieren información reciente de eventos en tiempo real) y valoran el feature versioning automático que permite reproducir exactamente qué features se usaron en un training específico.

**Tecton** es la opción con el mayor nivel de abstracción operativa y las capacidades más avanzadas para producción a gran escala. Sus diferenciadores son la real-time feature computation con SLA de sub-100ms (usando infraestructura gestionada de Redis y Kafka que Tecton opera internamente), el backfilling automático de features históricas cuando se añade una nueva feature definition (sin necesidad de ejecutar manualmente un job de backfill), y el feature monitoring nativo que detecta drift en las distribuciones de features y alerta antes de que el impacto llegue a los modelos. El costo de Tecton es significativamente mayor que las alternativas open source —con precios enterprise basados en volumen de features y requests— pero para organizaciones donde el equipo de plataforma tiene capacidad limitada para operar infraestructura compleja, el menor esfuerzo operativo puede justificar el diferencial de costo.

## Comparación técnica

- **Feast:** open source (Linux Foundation), deployment self-managed en Kubernetes, soporte para batch y streaming features via Kafka, bajo costo de licencia pero mayor esfuerzo operativo del equipo de plataforma; para organizaciones con capacidad operativa de Kubernetes.
- **Hopsworks:** open source con Enterprise tier managed, motor de streaming Flink integrado, feature versioning nativo con `FeatureGroup.insert()`, estadísticas automáticas de distribución; para casos de uso con streaming features y requisitos de versioning.
- **Tecton:** SaaS propietario, real-time feature computation con SLA de sub-100ms, automatic backfilling de features históricas, feature monitoring nativo; mayor costo pero menor esfuerzo operativo; para organizaciones que priorizan velocidad de adopción y tienen presupuesto para SaaS.
- **Criterio de selección por esfuerzo operativo:** Feast si el equipo puede y quiere operar infraestructura; Hopsworks si se necesita streaming y versioning sin asumir toda la operación; Tecton si la velocidad de time-to-value supera la consideración de costo.
- **Integración con el model registry:** los tres soportan integración con MLflow para registrar automáticamente las features usadas en cada training run como parte de los

metadatos de linaje; esto debe configurarse explícitamente, no es automático por defecto.

**Nota del Arquitecto:** Antes de elegir un feature store, el ejercicio más valioso es identificar los dos o tres casos de uso de ML en producción que más se beneficiarían de él y hacer un proof of concept con la solución candidata sobre esos casos específicos. La comparación de features en la documentación nunca captura el problema de integración real con los sistemas de datos existentes, la curva de aprendizaje del equipo, o los edge cases del caso de uso específico. El PoC de dos semanas ahorra meses de frustración post-adopción.

La elección entre Feast, Hopsworks y Tecton determina cuánto esfuerzo operativo asume el equipo de plataforma, no qué calidad de features pueden servirse. La siguiente sección cambia de dominio: de los feature stores para modelos de ML clásico a los pipelines de datos especializados para LLMs, que operan en escalas y con problemas de calidad fundamentalmente distintos.

---

## Módulo 10 – Capítulo 04 – Sección 03

---

### Pipelines de datos para LLM: ingestión, limpieza y preparación de datos de entrenamiento

A partir de esta sección, el capítulo cambia de dominio: de los feature stores para modelos de ML clásico a los pipelines de datos para LLMs. Este cambio no es menor. Los feature stores gestionan features estructuradas con latencias de milisegundos para inferencia en tiempo real; los pipelines de datos para LLMs procesan texto no estructurado a escala de terabytes o petabytes para producir corpora de entrenamiento. Las herramientas, las métricas de calidad y los desafíos operacionales son fundamentalmente distintos. Un ML Engineer con experiencia en feature stores para modelos tabulares encontrará que los pipelines de LLM requieren aprender un conjunto de técnicas diferente, aunque el principio subyacente sea el mismo: garantizar que el modelo se entrena con datos de alta calidad.

Los pipelines de datos para LLMs difieren de los pipelines de ML clásico en dos dimensiones: escala y complejidad de las transformaciones. En escala, un corpus de preentrenamiento de un LLM de tamaño medio procesa entre 100 billones y 1 billón de tokens (entre 100B y 1T tokens), lo que implica decenas o cientos de terabytes de texto. Mientras un dataset de ML tabular puede procesarse en horas con Spark sobre un cluster moderado, un corpus de preentrenamiento requiere pipelines distribuidos que corren durante días sobre cientos de nodos. En complejidad de transformaciones, los pipelines de LLM incluyen etapas de limpieza específicas para texto que no tienen equivalente en ML clásico: deduplicación fuzzy con MinHash LSH para detectar documentos parecidos aunque no idénticos, identificación de idioma a nivel de documento con fastText, evaluación de la calidad del contenido mediante clasificadores de perplexidad entrenados sobre texto de referencia de alta calidad, y filtrado de contenido dañino con listas de bloqueo y clasificadores.

El pipeline de datos para **fine-tuning de LLMs** (instruction tuning, RLHF, RLAIIF) opera en una escala más manejable —típicamente decenas de miles a pocos millones de ejemplos— pero con requisitos de calidad más estrictos. El dataset de instruction tuning, compuesto por pares prompt/completion, necesita pasar por validaciones de formato (¿el par tiene la estructura de sistema/usuario/asistente correcta?), filtrado de respuestas de baja calidad

(respuestas demasiado cortas, respuestas en el idioma incorrecto, respuestas que no siguen el formato esperado), deduplicación exacta (hash SHA256 de la muestra normalizada para eliminar duplicados idénticos que producen overfitting), y opcionalmente verificación de calidad con un modelo juez. El **LLM-as-a-judge** —usar GPT-4 o Claude para evaluar la calidad de cada par de training en una muestra representativa antes de incluirlo en el dataset — es una técnica con costo no trivial pero que puede identificar sistemáticamente respuestas incorrectas, inseguras o de baja calidad que los filtros heurísticos no detectan.

Las herramientas especializadas para este dominio incluyen: `datatrove` de Hugging Face para pipelines de limpieza de texto a escala con componentes reutilizables; `datasets` de Hugging Face para gestión de datasets con caching inteligente que evita recomputar etapas ya completadas; EleutherAI's `lm_datasets` para procesamiento de Common Crawl con filtros pre-calibrados; y Apache Beam sobre Dataflow (GCP) o Spark sobre EMR (AWS) para el procesamiento distribuido de las etapas más computacionalmente intensivas. El formato de almacenamiento para training eficiente es importante: MosaicML MDS (Mosaic Data Shard) y el formato nativo de Hugging Face datasets son los estándares para carga eficiente durante el entrenamiento con acceso aleatorio y sin cuello de botella de I/O.

## Etapas técnicas del pipeline de datos para LLM

- **Ingestión:** descarga y normalización de fuentes heterogéneas (Common Crawl WARC, GitHub, arXiv, Wikipedia dumps) con conversión a formato estándar JSONL con campos `text`, `source`, `timestamp`, `url`; la estandarización del formato es el primer paso de la cadena de calidad.
- **Limpieza de texto:** eliminación de HTML/boilerplate (Trafilatura, Resiliparse), normalización Unicode, detección de idioma (fastText `lid.176.bin`), y filtros heurísticos de calidad (longitud mínima, ratio de caracteres especiales, ratio de tokens alfabéticos).
- **Deduplicación:** exacta por hash del texto normalizado para eliminar copias idénticas; fuzzy con MinHash + LSH (similitud de Jaccard  $> 0.8$  entre documentos) que detecta near-duplicados que la deduplicación exacta no identifica; esta etapa reduce corpora en un 10-40%.
- **Filtrado de calidad:** perplexity scoring con KenLM entrenado sobre texto de alta calidad (Wikipedia), clasificadores de calidad curated vs. web data, y listas de bloqueo de dominios de baja calidad; estas etapas son las que más diferencian la calidad final del corpus.
- **Tokenización y empaquetado:** tokenización con el tokenizer del modelo objetivo, empaquetado en secuencias de longitud fija (2048 o 4096 tokens) con document



boundaries marcados, almacenamiento en formato MDS o Hugging Face datasets para carga eficiente durante el entrenamiento sin cuello de botella de I/O.

**Nota del Arquitecto:** *La calidad del corpus de entrenamiento determina más el comportamiento del LLM que la arquitectura del modelo. Una hora invertida en mejorar los filtros de calidad del pipeline —calibrar mejor el umbral de perplexity, añadir un clasificador de dominio específico, refinar la lista de dominios de baja calidad— tiene mayor impacto en el modelo final que una hora ajustando hiperparámetros de entrenamiento. Los equipos que invierten en la calidad del pipeline de datos iterativamente, eval por eval, son los que producen los modelos más consistentemente buenos.*

Los pipelines de datos para LLMs requieren infraestructura distribuida, herramientas especializadas para texto a escala, y métricas de calidad distintas de las del ML clásico. La siguiente sección conecta los dos dominios cubiertos en este capítulo con el tema del versionado de datos: tanto los features de ML clásico como los corpora de LLMs necesitan versionado reproducible para garantizar que los experimentos sean comparables y auditables.

---

## Módulo 10 – Capítulo 04 – Sección 04

---

### Data versioning: DVC y LakeFS para reproducibilidad de experimentos

El versionado de datos es tan crítico para la reproducibilidad como el versionado del código fuente, pero su implementación es más compleja porque los datasets de ML tienen propiedades que los sistemas de control de versiones de código como Git no pueden gestionar directamente: son binarios de gran tamaño (gigabytes o terabytes), cambian frecuentemente, y necesitan soportar consultas de "dame los datos que existían el 15 de marzo a las 10:00 AM" para reproducibilidad exacta de experimentos. La consecuencia de no tener versionado de datos es concreta: un mismo código de entrenamiento aplicado sobre datos distintos produce modelos distintos, y sin versionado de datos es imposible diagnosticar regresiones de calidad, reproducir experimentos de hace varios meses, ni cumplir con requisitos de auditoría que exigen saber exactamente qué datos de clientes fueron usados en cada modelo. Esta aplicabilidad es transversal a los dos dominios del capítulo: tanto los feature sets para ML clásico como los corpora de LLMs necesitan versionado.

**DVC** (Data Version Control) extiende Git para versionar datasets y modelos de gran tamaño sin introducir los archivos binarios en el repositorio de Git. Su mecanismo es elegante: al ejecutar `dvc add data/train.parquet`, DVC crea un archivo `data/train.parquet.dvc` con el hash MD5 del contenido (el archivo "puntero") y añade el dataset real al `.gitignore`. El puntero se versiona en Git normalmente — `git commit -m "update training dataset"`, `git push` — y el dataset real se almacena en un remote storage configurado (S3, GCS, Azure Blob, SSH) con `dvc push`. Para reproducir exactamente un experimento del pasado, `git checkout <commit-id>` restaura el puntero histórico y `dvc pull` descarga exactamente el dataset que ese commit referencia. DVC también gestiona pipelines de transformación con `dvc.yaml`: cada etapa del pipeline tiene entradas, salidas y comandos declarados, y `dvc repro` ejecuta solo las etapas cuyos inputs han cambiado, similar al comportamiento de `make`.

**LakeFS** adopta un enfoque arquitecturalmente diferente: en lugar de añadir una capa sobre Git, actúa como un sistema de control de versiones Git-like directamente sobre object storage (S3, GCS, Azure Blob) sin necesitar copiar los datos. LakeFS usa content-addressable storage

para crear referencias (branches, commits, tags) que apuntan a los mismos objetos subyacentes con costo cero de branching: crear un branch en LakeFS es instantáneo independientemente del tamaño del data lake porque no copia ningún dato, solo crea una referencia. `lakectl commit` crea un snapshot atómico de millones de archivos en milisegundos. `lakectl checkout <commit-id>` restaura exactamente el estado del data lake en cualquier momento del pasado. Esta arquitectura hace que LakeFS sea especialmente adecuado para organizaciones que ya tienen grandes data lakes en S3 y quieren añadir versionado sin migrar los datos existentes.

La elección entre DVC y LakeFS depende del contexto de uso y la escala. DVC está optimizado para repositorios de experimentos individuales o de equipo: funciona bien cuando el dataset de un experimento es el foco del versionado y el data lake completo no necesita ser versionado. LakeFS está optimizado para data lakes organizacionales completos: cuando múltiples pipelines y equipos comparten el mismo data lake y se necesita versionado de todo el estado del lake, no solo de datasets individuales. Muchas organizaciones usan ambos: LakeFS para el data lake y DVC para los datasets de entrenamiento específicos de cada experimento de ML.

## Aspectos técnicos del versionado de datos

- **DVC tracking:** `dvc add` crea el puntero con hash MD5; `dvc push` sube al remote storage; `dvc pull` restaura la versión exacta; el grafo de dependencias de `dvc.yaml` hace que `dvc repro` sea determinista e incremental.
- **DVC pipelines:** `dvc.yaml` define el grafo de transformaciones con inputs, outputs y comandos; `dvc repro` ejecuta solo las etapas cuyos inputs cambiaron, convirtiendo el pipeline de datos en un sistema reproducible y eficiente.
- **LakeFS branching:** cada experimento puede trabajar en su propio branch del data lake ( `feature/experiment-42` ) con aislamiento completo de las modificaciones hasta hacer merge; múltiples experimentos en paralelo sin riesgo de interferencia.
- **LakeFS commits atómicos:** `lakectl commit` captura el estado de millones de archivos usando content-addressable storage sin copiar datos físicamente; el costo de un snapshot es independiente del tamaño del data lake.
- **Integración con linaje de modelos:** DVC ( `dvc.lock` con hashes exactos) y LakeFS (commit SHA del data lake) proveen los identificadores únicos que se registran en el model registry como parte del linaje del modelo, cerrando el ciclo de trazabilidad datos → modelo.

**Nota del Arquitecto:** Registrar el hash del dataset (DVC) o el commit del data lake (LakeFS) junto con el `run_id` de MLflow en cada experimento crea automáticamente el linaje completo datos-experimento-modelo sin esfuerzo adicional. La clave es hacer este registro automático en el código base del pipeline de entrenamiento, no opcional para cada ML Engineer. La trazabilidad que no es automática no existe en producción.

El versionado de datos es la base técnica de la reproducibilidad: sin él, los experimentos son eventos únicos que no pueden repetirse ni compararse con rigor. La sección siguiente aborda la dimensión continua del problema de calidad de datos: no solo versionar los datos, sino monitorear que su calidad se mantiene en el tiempo y que los pipelines detectan degradaciones antes de que afecten a los modelos en producción.

---

## Módulo 10 – Capítulo 04 – Sección 05

---

### Data quality: validación y monitoreo de calidad de los datos que alimentan el sistema

La calidad de datos en un sistema de IA no es un estado que se alcanza una vez durante el desarrollo del pipeline y se mantiene automáticamente: es una propiedad que puede degradarse en cualquier momento por razones que van desde bugs en sistemas upstream hasta cambios de comportamiento de los usuarios que producen distribuciones de datos no anticipadas. El monitoreo de calidad de datos es el proceso continuo que detecta estas degradaciones estadísticas antes de que impacten la calidad de los modelos, actuando como la primera línea de defensa del sistema de IA. Esta detección temprana es económicamente decisiva: un problema de calidad de datos detectado en el pipeline antes de llegar al modelo se resuelve en horas; el mismo problema detectado después de que el modelo degradado lleva semanas en producción puede requerir un reentrenamiento de emergencia, una investigación de incidente y comunicación a equipos de negocio.

**Great Expectations** es el framework estándar de la industria para definir y ejecutar validaciones de calidad de datos. Su modelo conceptual es simple pero poderoso: cada dataset tiene un conjunto de "expectations" que son afirmaciones sobre las propiedades que los datos deben satisfacer, y Great Expectations ejecuta esas afirmaciones contra los datos reales y reporta qué expectations pasaron y cuáles fallaron. Las expectations se definen en Python de forma declarativa: `expect_column_values_to_be_between("age", 0, 120)` verifica que todos los valores de la columna `age` estén en el rango esperado; `expect_column_proportion_of_unique_values_to_be_between("user_id", 0.95, 1.0)` verifica que al menos el 95% de los valores de `user_id` sean únicos; `expect_column_distribution_to_match_other("feature_x", reference_dataset)` compara la distribución de `feature_x` contra una distribución de referencia. Estas validaciones se ejecutan automáticamente en cada run del pipeline de datos, y si alguna falla, el pipeline puede detenerse antes de usar datos inválidos para entrenamiento o inferencia.

El monitoreo continuo en producción requiere un enfoque diferente de las validaciones batch del pipeline: en lugar de ejecutar validaciones sobre el dataset completo periódicamente, el

monitoreo continuo calcula estadísticas de distribución sobre ventanas deslizantes de datos de producción y las compara contra una distribución de referencia. Los tests estadísticos estándar para este monitoreo son: **Kolmogorov-Smirnov** (KS-test) para variables continuas, que mide la máxima diferencia entre la CDF de la distribución actual y la de referencia; **Chi-squared** para variables categóricas, que mide si la frecuencia de cada categoría es consistente con la distribución de referencia; y **Population Stability Index (PSI)**, que combina ambas distribuciones en un índice scalar con umbrales bien establecidos:  $PSI < 0.1$  indica estabilidad,  $0.1-0.2$  indica drift moderado que merece atención, y  $PSI > 0.2$  indica drift severo que señala la necesidad de reentrenamiento del modelo.

Una complicación práctica del monitoreo estadístico es el problema de las comparaciones múltiples: si se monitorizan 100 features y se aplica un umbral de significancia de  $p < 0.05$  a cada una, se esperan aproximadamente 5 alertas falsas por ciclo de evaluación incluso si los datos no han cambiado. La corrección de Bonferroni —dividir el umbral de significancia por el número de features monitorizadas— reduce los falsos positivos pero puede hacer que el sistema sea insensible a drift real en datasets con muchas features. Una solución práctica es jerarquizar las features: las features críticas para el modelo (las que tienen mayor importancia en SHAP values o feature importance) se monitorizan con umbrales más sensibles y alertas inmediatas; las features secundarias se monitorizan con umbrales más permisivos o se agregan en un score de drift global.

## Dimensiones de calidad de datos para sistemas de IA

- **Complejidad:** porcentaje de valores nulos por columna; alertar si supera el umbral definido (ej.  $>5\%$  para features críticas,  $>20\%$  para features secundarias); un aumento repentino de nulos suele indicar un bug en el sistema upstream.
- **Frescura:** tiempo desde la última actualización de las features en el online store; la latencia de materialización se mide y alerta cuando supera el SLO; features desactualizadas pueden degradar el modelo silenciosamente.
- **Distribución:** drift estadístico medido con KS-test (variables continuas,  $p < 0.05$ ) o Chi-squared (variables categóricas), ejecutado periódicamente comparando datos de producción contra distribución de referencia (el dataset de entrenamiento).
- **Esquema:** validación automática de que el schema (columnas, tipos, cardinalidades) coincide con el contrato definido; cambios de schema se detectan inmediatamente antes de que rompan el pipeline de features.

- **Consistencia referencial:** validación de que los identificadores (user\_id, item\_id) en los datos de features existen en los sistemas fuente y no hay data corruption por JOINS incorrectos o deletes no propagados.

**Nota del Arquitecto:** La validación de datos debe ejecutarse en cada etapa del pipeline, no solo al inicio. Datos que entran al pipeline con el schema correcto pueden corromperse en transformaciones intermedias —un join con NULL behavior incorrecto, una agregación con lógica de time window equivocada, un encoding de categorías que produce valores fuera de rango. La detección en la etapa siguiente de la transformación incorrecta, no al final del pipeline, es lo que permite identificar la causa raíz con precisión.

La validación y el monitoreo de calidad de datos son las garantías técnicas de que el sistema de IA recibe inputs correctos. Sin ellas, la calidad de los modelos en producción es un evento aleatorio que depende de que los sistemas upstream no cambien de forma inesperada. La sección de cierre de este capítulo integra el argumento central: la inversión en infraestructura de datos —feature stores, versionado, validación— es siempre rentable cuando se mide correctamente como el costo evitado de incidentes de producción causados por datos de baja calidad.

## Módulo 10 – Capítulo 04 – Sección 06

---

### **Cierre: los datos son el input más importante del sistema — su calidad determina todo lo demás**

Los feature stores, los pipelines de datos para LLMs, el versionado con DVC o LakeFS, y los frameworks de validación como Great Expectations no son componentes opcionales de una plataforma de IA madura: son la garantía de que los modelos se entrenan con datos correctos, consistentes y reproducibles, y de que los sistemas de inferencia reciben en producción la misma calidad de datos que el modelo vio durante el entrenamiento. Este conjunto de garantías es la diferencia entre un sistema de IA cuyo comportamiento se puede explicar y auditar, y uno cuyo comportamiento es opaco incluso para los equipos que lo construyeron.

El training-serving skew —la divergencia entre los datos de entrenamiento y los datos de inferencia— es una de las causas más frecuentes y más difíciles de diagnosticar de degradación de modelos en producción, precisamente porque no produce errores de sistema: el endpoint responde correctamente desde el punto de vista de la infraestructura mientras el modelo produce predicciones silenciosamente peores. El feature store es la solución estructural a este problema porque garantiza que la misma lógica de cálculo produce las mismas features en ambos contextos; sin él, las organizaciones dependen de la disciplina de los equipos individuales para mantener la consistencia, y esa disciplina falla inevitablemente bajo presión operativa.

Un pipeline de datos bien construido es también el primer nivel de defensa contra incidentes de IA: cuando los datos de entrada a un modelo cambian de forma inesperada —un campo que empieza a llegar nulo, una distribución que cambia por un bug en el sistema de origen, un cambio de schema no comunicado por un equipo upstream— el pipeline de validación debe detectar ese cambio antes de que el modelo procese esos datos y produzca predicciones incorrectas. Esta detección temprana convierte lo que podría ser un incidente de producción silencioso en una alerta de pipeline que el equipo puede investigar y resolver en horas, sin impacto visible para los usuarios.



El valor de la infraestructura de datos se hace visible en retrospectiva: las organizaciones que invirtieron en feature stores, versionado de datos y validación automática durante los primeros años de sus sistemas de IA son las que pueden responder rápidamente a preguntas de auditoría, reproducir experimentos de hace años, e investigar incidentes de producción con trazabilidad completa. Las organizaciones que no lo hicieron están, años después, tratando de reconstruir retroactivamente qué datos usaron en qué modelos, con la urgencia de auditorías regulatorias o de incidentes de producción que exigen respuestas que el sistema no puede proveer.

## Principio rector

La inversión en calidad de datos es siempre rentable cuando se mide correctamente: cada hora dedicada a construir validaciones de datos robustas evita días de investigación de incidentes de producción causados por datos incorrectos. El primer nivel de gobierno de un sistema de IA no está en el modelo ni en el serving layer —está en la calidad y la trazabilidad de los datos que alimentan el pipeline.

---

*"Torture the data, and it will confess to anything."*

— Ronald Coase, Premio Nobel de Economía, advirtiéndole sobre el riesgo de extraer conclusiones incorrectas de datos de baja calidad o mal procesados.